Moving the Mesh Without Remaking It

Louis Moresi [1]

[1] Australian National University

Keywords Underworld Code, Tricks of the Trade, development

A geodynamic model spends most of its resolution in the wrong place. The features that need small cells — a thermal boundary layer, a shear band, a subducting slab's nose, a fault — occupy a few percent of the domain and move around during the calculation. A uniform mesh sized for the thinnest of them is sized for all of it, and most of that expense buys nothing.

The standard answer is adaptive refinement: mark the cells where an error estimate is large, split them, repeat. It works, and it is the right tool often enough that every serious code has it. But it charges for the resolution twice, and the second charge is the one nobody quotes.

1. What refinement charges

Refinement changes the point set, and in a parallel run three things follow.

The partition is no longer balanced. New cells appear where the physics is interesting, which is to say unevenly, so the ranks that own the interesting region end up owning most of the work. Fixing that means repartitioning and migrating — cells, degrees of freedom, every field, and any particles riding along. That migration is not a detail of the implementation. It is communication proportional to how much the mesh changed, and it happens every time the mesh changes.

Fields have to be transferred between meshes that share no nodes. The old and new meshes are related by the refinement rule, but the values live at places that do not correspond, and interpolating between them costs accuracy every time. Do it every ten steps for a thousand steps and the transfer error is a term in your answer.

The multigrid hierarchy is not what it was. This is the one worth spelling out, because it is easy to assume that refinement builds a hierarchy for free — after all, the parent mesh is right there.

Full multigrid gets its efficiency from levels separated by a factor of two in resolution. Each level is responsible for the error at its own wavelength, and the smoother on that level removes it cheaply. Adaptive refinement does not produce levels like that. A pass that marks five percent of the cells produces a mesh whose *mean* spacing differs from its parent's by a couple of percent. As a multigrid level, it costs a smoother sweep on every cycle and removes almost nothing, because there is no band of error that only it can see. Run seven adaptation

PUBLISHED Aug 12, 2026 ARTICLE UWTN 2026-011 VERSION 1.0.0 LICENCE CC-BY-4.0 UNDERWORLD underworld3 0.0.0 SOURCE https://www.underworldcode.org/moving-the-mesh-without-remaking-it/

passes and you have seven such levels and a solver slower than the one you started with. On the cut fault meshes where we first ran into this, throwing away the levels that were not a genuine doubling of h made the Stokes solve about four times faster.

So the hierarchy that survives an adaptation is not the hierarchy you want, and building the one you do want means starting over: coarsen, rebuild the transfers, redistribute, and hope the coarse levels still represent the fine problem.

2. Move the nodes instead

Here is the alternative, and the whole of this note is about what happens when you take it seriously.

What if the node budget is fixed? No node is created, none is destroyed, and no cell changes its neighbours. Every node stays on the rank that owned it. What moves is where the nodes **are** — they slide to where the resolution is wanted and away from where it is not.

Everything that made refinement expensive is then absent by construction:

- the partition is unchanged, because ownership is a property of the point set and the point set did not change;
- the connectivity is unchanged, so the multigrid hierarchy — which is built from the topology, not from the coordinates — is still a hierarchy;
- there is no mesh-to-mesh transfer, because there is only one mesh.

What you give up is equally clear, and we will come back to it: a fixed budget of nodes can only be redistributed, never increased.

3. It is an equidistribution problem, not a smoothing heuristic

Node movement has a bad name, and deservedly, because most of it is smoothing: push each node toward the average of its neighbours, iterate, stop when it looks tidier. Smoothing has no target. It cannot tell you when it is finished and it cannot be aimed at the physics.

The formulation that can is *equidistribution*. Supply a metric $M(x)$ — a monitor function saying how much resolution is wanted, and in a tensor metric, in which direction — and ask for the map from a fixed computational mesh to the physical mesh under which every cell carries the same amount of M. That is a Monge–Ampère-style problem, and it is the same target the refiner is chasing; the difference is only that the refiner is allowed to add nodes and this is not.

Underworld3 solves it with the Huang–Kamenski moving mesh PDE (Huang & Kamenski, 2015), which generates the physical mesh as the image of a fixed computational mesh under the inverse coordinate map, minimising

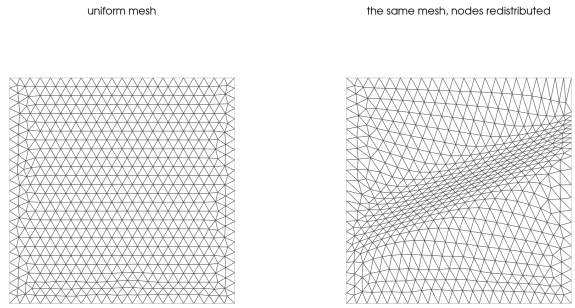
$$G = \theta\sqrt{\det M}\, S^{q} + (1-2\theta)\, d^{\,q}\, r^{p} (\det M)^{(1-p)/2}, \qquad q = \frac{d\,p}{2}, \tag{1}$$

with $S = \operatorname{tr}\!\left(\mathbb{J}\, M^{-1} \mathbb{J}^{T}\right)$, $\mathbb{J} = \hat{E} E^{-1}$ the Jacobian of the map between the reference and physical elements, and $r = \det \mathbb{J}$. The first term is the shape term, the second the size term.

Two properties of that functional are why this is the mover we use and not one of the several we retired.

It cannot fold the mesh. $G \to \infty$ as $\det \mathbb{J} \to 0$, so a cell cannot be driven through zero volume without the energy going to infinity first. That is a theorem about the functional (Huang & Kamenski, 2017), not a guard bolted on afterwards, and it is the difference between a mover you can leave running in a time loop and one you have to watch.

It aligns as well as clusters. M is a tensor, so a metric that is thin across a fault normal and long along it produces cells that are thin across the fault and long along it. An isotropic monitor function can only ask for smaller cells; a tensor one can ask for the *right* cells, which for a sheet-like feature is worth far more than the same node count spent isotropically.




```
  is uniform. On the right the same mesh has
its cells drawn in along a
  diagonal band, where they are noticeably
smaller and elongated along the
  band direction, and correspondingly larger
away from it.
```

(a) (b)

Figure 1: The same mesh, before and after redistribution to an anisotropic metric across a diagonal band. Both panels have **1152 cells and 621 nodes** — the counts are not merely similar, they are the same mesh object at two moments. What changed is where the nodes are: the median cell on the band is 1.65× smaller than in the bulk, and the cells there are visibly stretched along the band rather than simply shrunk. Nothing was refined, nothing was transferred between meshes, and no node changed rank.

In use it is two calls:

```python
import underworld3 as uw

# A scalar metric from |grad T|. refinement=R is the finest:coarsest
# cell-size ratio you are asking for, not a quality target.
rho = uw.meshing.metric_density_from_gradient(
    mesh, T, refinement=5, coarsening="auto",
    metric_choice="front-following")

# Move the nodes. The mover owns field transfer: it remaps T, carries
# the semi-Lagrangian history and fires the on_remesh hooks.
uw.meshing.node_redistribution(
    mesh, rho,
    method_kwargs=dict(step_frac=0.2, accel="none", momentum=0.0,
                       n_outer=400),
    slip_surfaces=True,      # boundary nodes slide tangentially
    skip_threshold=0.9)      # don't move an already-aligned mesh
```

`accel="cg"` currently under-delivers

The conjugate-gradient accelerator is the documented default, and while producing the figure above we found that it stalls: the line search rejects the accelerated direction, backtracks to a zero step, and the outer loop reads that as convergence. On the problem shown it stopped at iteration 9 of 150 having reached 1.08× grading, where the unaccelerated mover reaches 1.65×. Tolerances, `n_outer` and the area floor are

all uninvolved — only the accelerator matters. Until underworld3#531 is closed, pass `accel="none"` and a larger `n_outer` when the grading you get looks weaker than the grading you asked for.

4. The transfer that remains, and why it is cheap

It would be too neat to say no field transfer is needed. Nodes move, so the value stored at a node is now the value of the old field at a place the node no longer is, and it has to be re-evaluated. That is an interpolation and it costs accuracy like any other.

What it does not cost is communication. A node moves a fraction of a cell diameter, so the point it must be evaluated at lies in its own patch or a neighbour's — inside the halo the mesh already maintains. Compare the remeshing case, where source and target are unrelated meshes and locating a point can land anywhere in the domain, on any rank, requiring a parallel search and then a migration to answer it. Same operation in name; entirely different in cost and in who has to talk to whom.

5. The reference frame is the lever

Now the part that changed what this mover is for.

The functional measures a cell's distortion against a *reference* element. By default, the reference is the mesh as it was on the first call. That is the correct choice for redistribution — you are asking for a map from this mesh to a better-graded one — and it has a consequence that is easy to miss until it bites.

Under a uniform metric, a distorted mesh is its own optimum. If $\hat{E} = E$ then $\mathbb{J} = I$ in every cell, the shape gradient is zero everywhere, and the mover moves nothing. Not approximately nothing: the measured displacement of `redistribute_nodes` under $M = I$ is exactly zero. Handed a mesh full of needles and asked to tidy it up, the redistributor correctly reports that it is already perfect.

The fix is not a different mover. It is a different reference. Replace each cell's reference element with a single *regular simplex*, scaled to that cell's own current volume, and the same functional means something else: "distorted" now means "away from equilateral" rather than "away from the mesh I was handed". Because each reference is scaled to the cell's own volume, $r = \det \mathbb{J} = 1$ at entry, so the size term starts at its optimum and stays there — the grading is preserved and only shape does work.

Three frames, then, from one piece of machinery:

`reference`	the reference element is	what it does
`"mesh"`	the mesh at first call	redistribute to a metric
`"ideal"`	a regular simplex at each cell's own volume	repair shape, hold size
`"ideal-metric"`	a regular simplex at one common volume	repair shape, metric sets size

The user-facing spelling is `mesh.relax()` for the metric-free case and `mesh.relax(M)` for the third, alongside `mesh.redistribute_nodes(M)` for the first.

`relax()` and `relax(metric)` are different operations

Metric-free relaxation is a shape guarantee: sizes are held, so shapes can only improve. Passing a metric re-grades the mesh, and re-grading will trade shape away to do it. On a four-level graded box the 99th-percentile maximum angle went 117.9° arrow.r 113.8° without a metric and 117.9° arrow.r **127.4**° with one. The metric version is not the better-informed version; it is a different job.

6. What this is worth on a refined mesh

Refinement chooses where a new node goes from *combinatorics*: which edge the tagging rule nominated under bisection, or the cell centroid under Alfeld. It never looks at geometry. So the needles and slivers a refined mesh carries reflect the base mesh's arbitrary choices rather than anything about the problem — which is exactly the situation the ideal frame was built for.

On a mesh from the library's own bisection adapt (five levels, 1143 cells), measuring the P1 interpolation error of a fault-localised field, relaxing once at the end cut the error by **17.5%** in serial and 17.2% on two ranks. No new cells, no new degrees of freedom, no change to the solver setup.

On the production path — gmsh base, one refinement pass, NVB adapt to three levels, Stokes preconditioned with a custom-prolongation full multigrid:

configuration	cells	error	error × cells	velocity KSP its
adapt only	832	2.746e-2	22.85	4
relax at end	832	2.559e-2	21.29	3
relax every generation	856	**2.384e-2**	**20.40**	4

The comparator is error × cells rather than error alone: P1 L_2 interpolation error goes like h^2 and cell count like h^{-2} in two dimensions, so their product is what does not depend on how fine you happened to go. Relaxing inside the refinement loop wins on both axes for three percent more cells.

Both placements are defensible and we deliberately do not rank them. `adapt(metric, relax=True)` relaxes once at the end and is the default, being cheaper and less invasive; `relax="per-generation"` relaxes inside the loop so each pass marks from already-relaxed coordinates, giving the cleanest elements and the lowest error at a small cost in cells.

How much this is worth depends on how bad the start is

The ordering study above used a structured right-triangle grid, which is nearly optimal for bisection to begin with ($q = 0.866$, maximum angle 90°), so the gains there are small. On a gmsh base — which is what production actually uses — the same operation was worth 17.5%. A mover's benefit scales with how much was wrong.

7. Does the hierarchy actually survive?

That is the claim the whole approach rests on, so it should be checked rather than asserted. In the production comparison above, all three configurations kept a full four-level `pc=mg`, none fell back to algebraic multigrid, all converged for the same reason, and their peak velocities agreed to four significant figures. Moving the nodes did not cost a level.

There is one real qualification. Multigrid does not care about coordinates — the hierarchy lives in the operators, and intermediate meshes are preserved as topology, not as geometry. But Underworld3's custom-prolongation transfers are *built* by geometric point location rather than derived from the refinement relation, and moving the nodes can leave a coarse degree of freedom with no fine image under the local-support barycentric builder. The build now retries with the global-support RBF builder before giving up. Measured in 3D: before the retry existed the hierarchy collapsed to algebraic multigrid at 23 iterations; with it, `pc=mg` at 2.

The topology survives, in other words, but a transfer operator built from geometry has to be told the geometry moved.

8. The ceiling: a fixed node budget

This is the honest limit, and it is structural rather than a matter of tuning.

A mover redistributes; it never adds a node. Starting from a uniform mesh it saturates — about 1.8× finer on a fault than in the background — and no amount of iteration goes past that. The nodes it would need are somewhere else, and if the metric has ridges between here and there, they cannot get here without someone paying in cell quality.

The remedy is not to iterate harder. It is to start with the nodes. Put the refinement into the gmsh base and the mover does not merely maintain it, it **compounds** it. Measured on a fault in an annulus, as the ratio of on-fault to bulk nearest-neighbour spacing — lower is finer:

base mesh	base ratio	after the mover
uniform	1.00	0.55 (∼1.8×)
gmsh, `refine_lines` at 2	0.44	0.29 (∼3.4×)
gmsh, `refine_lines` at 3	0.30	0.19 (∼5×)

The extra nodes lift the mover off its budget cap, and it then extracts grading the base mesh alone could not.

That ceiling is also the argument for the next note in this series. When the feature turns up where you did not put nodes at construction time — and in a convecting model it will — redistribution cannot reach it. Stacking local, ephemeral resolution *on top* of the moved mesh is the other answer, and it keeps every advantage described here underneath it.

9. What it does not fix

Three things, stated because each of them cost us a wrong conclusion first.

Slivers that are structural. Centroid and Alfeld refinement place an interior point against a fixed parent face. The resulting sliver is a property of the *topology*, and no amount of node motion repairs a topology. Relaxed or not, cells with $q < 0.3$ stay at 25–28% and maximum angles at 176–178°. The mover is a shallow tool on that mesh; use bisection.

Accuracy in three dimensions. In 3D, relaxation improves mesh quality substantially — the near-degenerate population halves (cells with $q < 0.1$, 3.6% arrow.r 1.8%), median quality goes 0.32 arrow.r 0.39, the 99th-percentile dihedral angle comes back from 153° to 146° — and leaves interpolation error alone (+0.5%). That is consistent rather than disappointing: relaxation holds the size distribution, and in 2D the accuracy gain came

from cells *aligning* onto the feature, which an isotropic metric in 3D gives them no reason to do. Use it in 3D for conditioning and element quality. An anisotropic 3D metric is the open question.

A single number for "wasted refinement". We wanted a scalar for how much resolution ends up outside the region that asked for it, and produced four, each wrong in its own way: one measured "the base is finer than the far-field target", one penalised smooth grading because a staircase scores well against it, one conflated helpful over-refinement on the feature with a useless halo off it, and one disagreed with what the mesh plainly looked like. The property is spatial — large coherent blocks of over-refinement are a problem, scattered patches are not — and every attempt collapsed it to a scalar. The deliverable is the per-cell map of $\log_2(h/h_{\text{asked}})$, and if a number is ever needed it should be the size of connected components, not a mean.

10. Two bugs worth naming

They are the same bug twice, in different places, and the pattern is worth more than either instance.

The line search that accepts a trial node position required the minimum cell area to stay above a floor — and the floor was one absolute value derived from the median cell volume across the whole mesh. On any graded mesh, which is to say on anything coming out of `mesh.adapt`, the finest cells start orders of magnitude below that floor. Every trial step was rejected, the line search backtracked to zero, and **the mover silently did nothing**.

Silently, and *partition-dependently*: the median was a global mean of per-rank medians, so the same mesh moved on one rank and stood dead still on two. A mover that does nothing produces a valid mesh and a plausible answer, which is why this survived as long as it did.

The fix is a per-cell relative floor — no cell may shrink below a fixed fraction of its *own* starting volume. Scale-free, grading-free, partition-independent, and still a strict no-fold certificate.

The second surfaced while making the figure in this note, and it is still open. The conjugate-gradient accelerator produces a search direction the line search will not accept; the line search backtracks to a zero step; and the outer loop reads a zero step as convergence and returns. Nine iterations out of a hundred and fifty, a quarter of the requested grading, and no complaint. Neither the tolerances nor the area floor are involved this time — only the accelerator — so the two defects share nothing but their symptom.

Which is the point. In both cases the mover returned a valid, non-folded mesh and a plausible answer, having quietly declined to do its job. The first survived long enough to make results partition-dependent; the second is underworld3#531 and had gone unnoticed because "the mover doesn't seem to do very much" reads as a modelling problem rather than a solver one.

Two lessons, then. Any threshold derived from a global average is a bug waiting for a mesh with a wide enough distribution. And a numerical component that fails by *doing nothing* is far more dangerous than one that fails by falling over: a zero step is not convergence, and code that cannot tell the difference will eventually tell you so at the worst possible moment.

11. Using it

```
# Redistribute to a metric: grading, in the mesh's own reference frame.
mesh.redistribute_nodes(metric)
```

```
# Repair element shapes at fixed size: the ideal frame.
child = mesh.adapt(metric, max_levels=3)
child.relax()

# Or ask adapt to do it for you.
child = mesh.adapt(metric, max_levels=3, relax=True)             # at the end
child = mesh.adapt(metric, max_levels=3, relax="per-generation") # in the loop
```

All of these keep the vertex count, the degree-of-freedom layout and the parallel partition exactly as they were. That is the point of the whole exercise: the resolution follows the physics, and everything built on top of the mesh — the multigrid hierarchy above all — does not have to be told.

References

Huang, W., & Kamenski, L. (2015). A geometric discretization and a simple implementation for variational mesh generation and adaptation. *Journal of Computational Physics*, *301*, 322–337. https://doi.org/10.1016/j.jcp.2015.08.032

Huang, W., & Kamenski, L. (2017). On the mesh nonsingularity of the moving mesh PDE method. *Mathematics of Computation*, *87*(312), 1887–1911. https://doi.org/10.1090/mcom/3271